
Experiment name: Simulation of Mealy and Moore Finite State Machines using Verilog in ModelSim



Department of Electrical and Electronic Engineering

Course Title: VLSI Design and Testing Lab

Course code: EEE 356

Group No: 12

EXP NO:06

Submitted By:

Name: Prapto Debnath

Registration No: 2021338016

Name: Sujon Bepary

Registration No: 2021338044

Dept. of Electrical & Electronic Engineering

Submitted To:

Myen Uddin

Lecturer

Department of Electrical & Electronic Engineering,
Shahjalal University of Science and Technology, Sylhet.

Date of submission

14-09-2026

Theory:

A Finite State Machine (FSM) is a digital system that works through a fixed number of predefined states. The system moves from one state to another depending on the input it receives and the clock signal. FSMs are widely used in digital electronics for tasks such as control operations, sequence detection, communication systems, and other applications where the system needs to follow a specific sequence of actions.

In an FSM, the clock acts like a timing signal that controls when the state can change, while the reset brings the system back to its initial state. The next state is decided based on the current state and the applied input. Depending on how the output is generated, FSMs are mainly divided into two types: Mealy and Moore machines.

More machine

In a Moore machine, the output depends only on the present state. This means the output does not change directly when the input changes. Instead, it changes when the clock causes the FSM to move to a new state. Because of this, the output is usually more stable and predictable. However, compared to a Mealy machine, a Moore machine may sometimes need an extra state or an extra clock cycle to produce the same output.

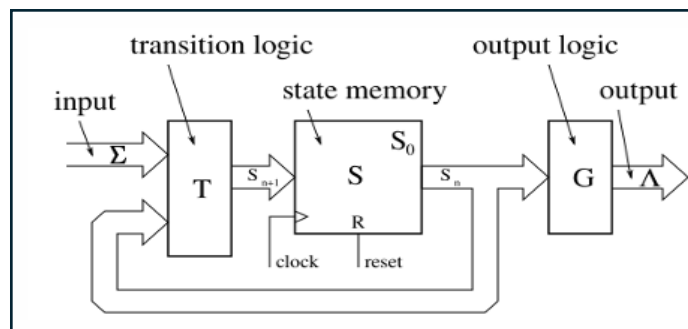


Fig: more machine

Detector (Moore) Overview:

1. Input: w (serial bit), sampled on rising clock edge
2. Control: synchronous clk , active-low asynchronous reset $\bar{n}$
3. Output: $z = 1$ when the machine is in the MATCH state

States (Moore outputs in brackets):

1. S0 – no relevant history yet [z=0]
2. S1 – last input was 1 [z=0]
3. S2 – detected "11" (match) [z=1]

Current	w=0	w=1	Output (z)
S0	S0	S1	0
S1	S0	S2	0
S2	S0	S2	1

Fig : state table of more machine

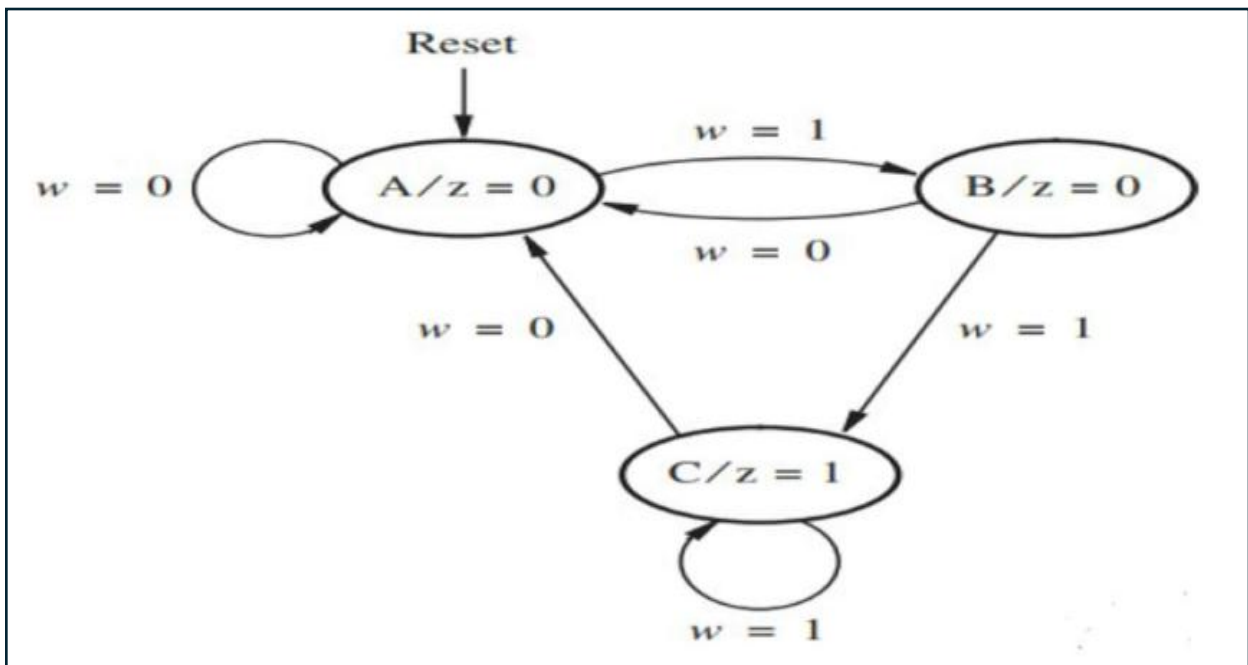


Fig :state diagram of more type machine

Mealy machine:

In a Mealy machine, the output depends on both the present state and the current input. So, when the input changes, the output can also change without waiting for the next clock edge. This makes the Mealy machine respond quickly and often allows it to use fewer states than a Moore machine. However, because the output is directly affected by the input, it can sometimes have small unwanted changes or glitches if the input signal is noisy. In the Mealy implementation, the output Z asserts high (logic 1) during the clock cycle where the second consecutive '1' arrives.

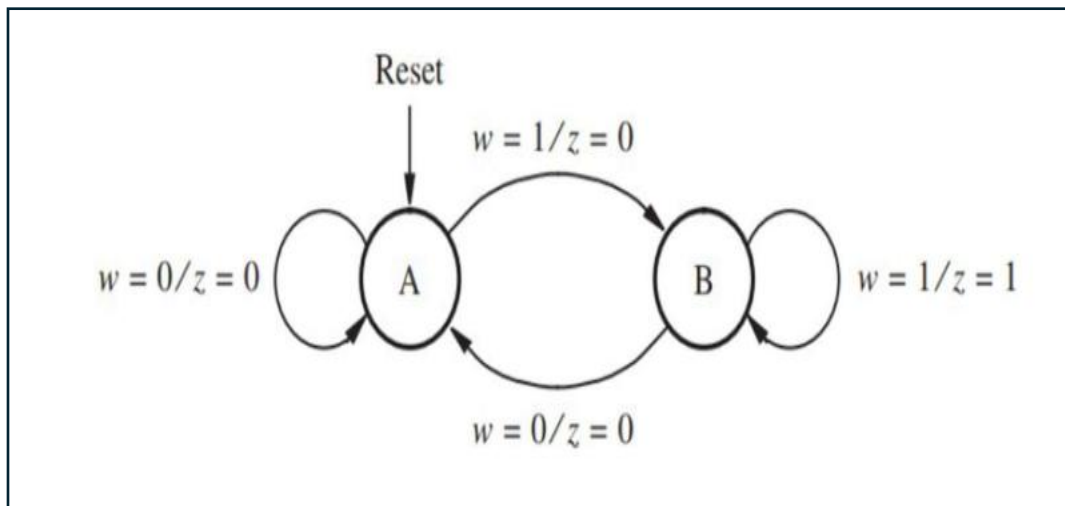
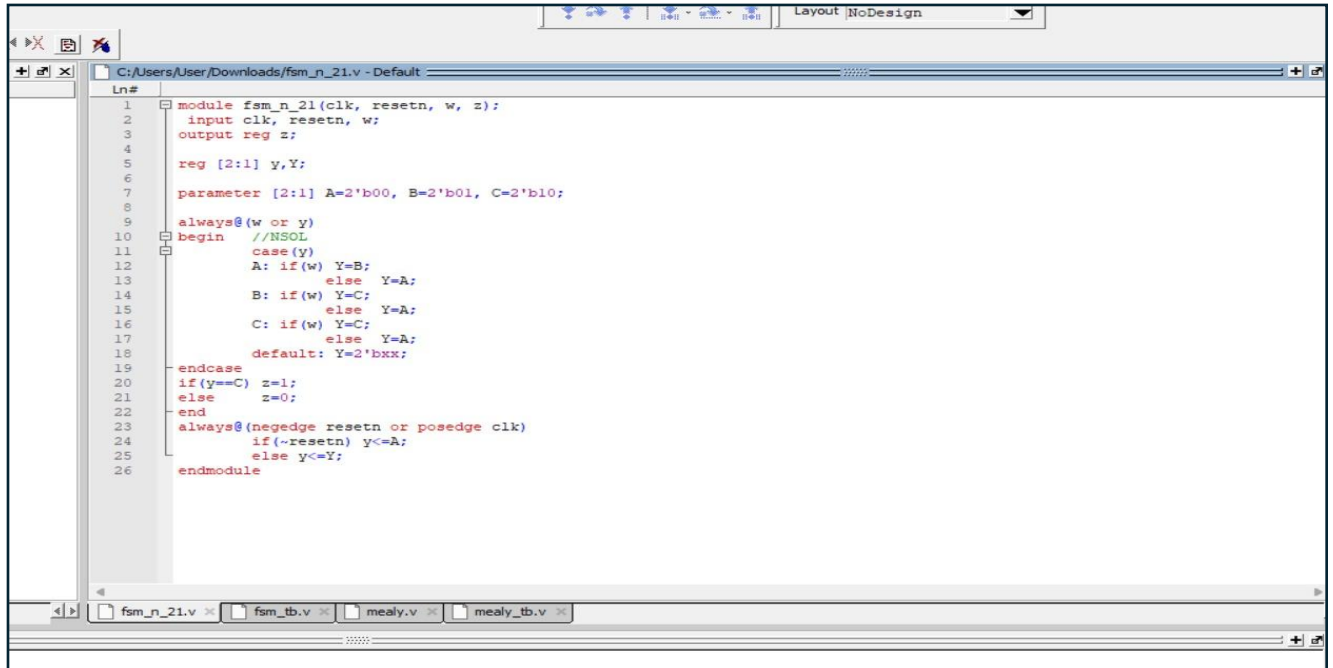


Fig: state diagram of melay machine

Present state	Next state		Output (z)	
	w=0	w=1	w=1	w=0
A	A	B	0	0
B	A	B	0	1

Fig : state table of melay machine

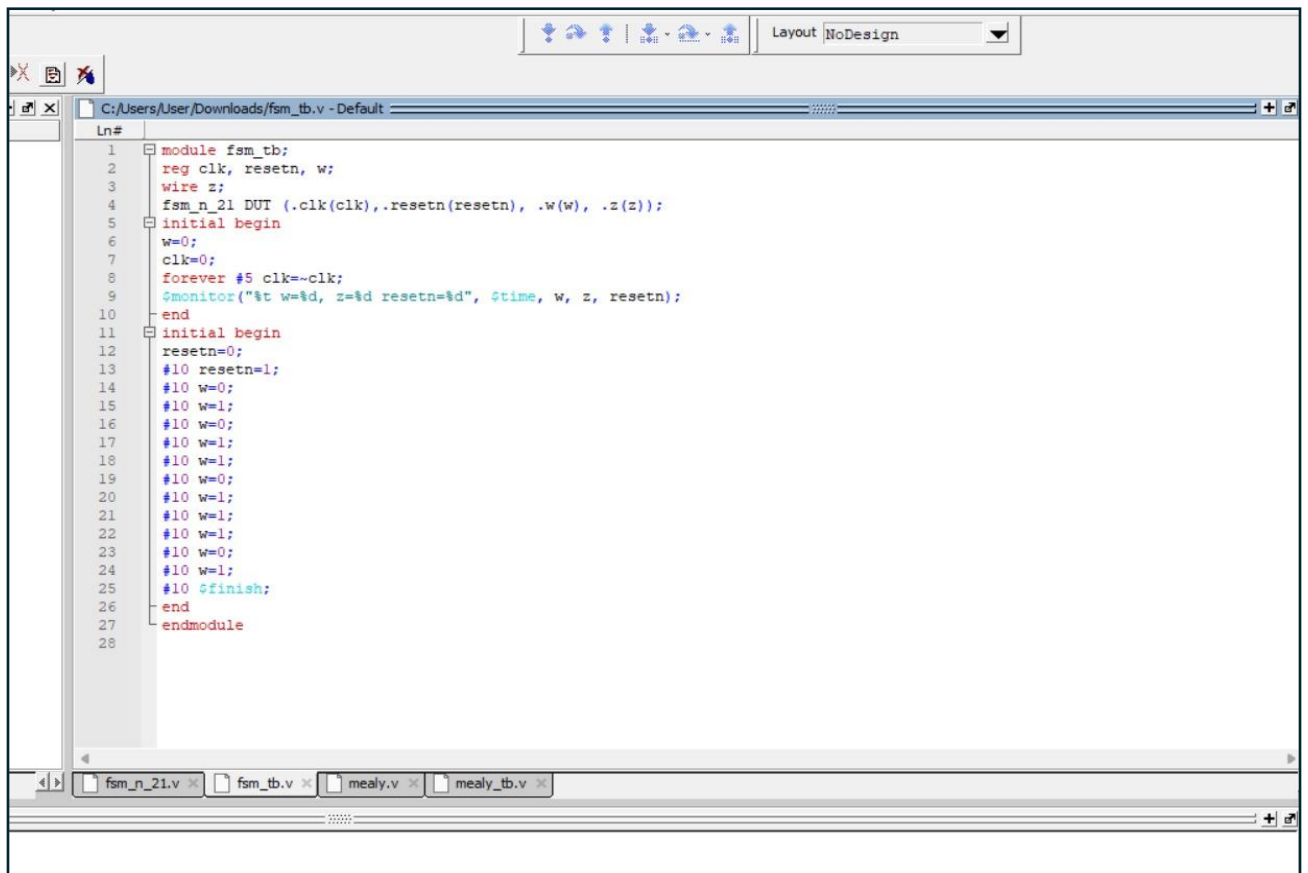
Verilog code:



```
1 module fsm_n_21(clk, resetn, w, z):
2   input clk, resetn, w;
3   output reg z;
4
5   reg [2:1] y,Y;
6
7   parameter [2:1] A=2'b00, B=2'b01, C=2'b10;
8
9   always@(w or y)
10    begin
11      //MSOL
12      case(y)
13        A: if(w) Y=B;
14           else Y=A;
15        B: if(w) Y=C;
16           else Y=A;
17        C: if(w) Y=C;
18           else Y=A;
19        default: Y=2'bxx;
20      endcase
21      if(y==C) z=1;
22      else z=0;
23    end
24    always@(negedge resetn or posedge clk)
25      if(~resetn) y<=A;
26      else y<=Y;
27  endmodule
```

The screenshot shows a Verilog code editor with the file path "C:/Users/User/Downloads/fsm_n_21.v - Default". The code defines a module fsm_n_21 with inputs clk, resetn, w and output reg z. It includes a 2-bit register y and a 2-bit parameter [2:1] A=2'b00, B=2'b01, C=2'b10. The logic is implemented using a case statement for the next state Y based on the current state y and input w. The output z is set to 1 when the current state is C, and 0 otherwise. The reset logic sets y to A when resetn is active low.

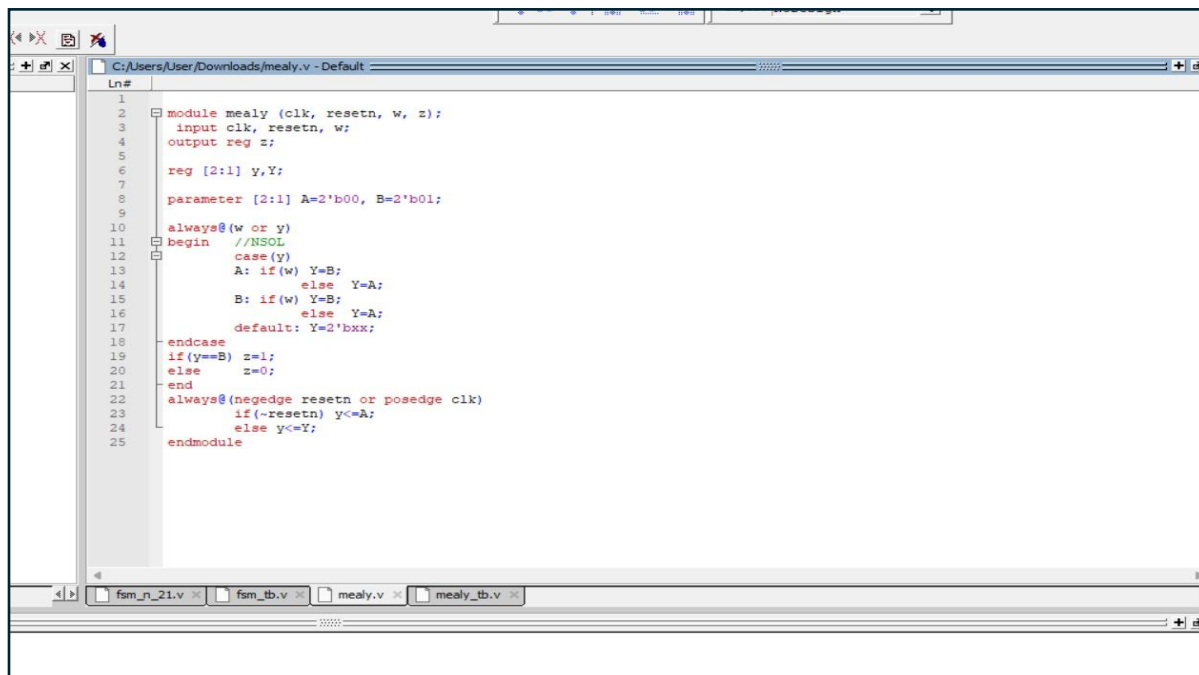
testbench code for moore type machine :



```
1 module fsm_tb;
2   reg clk, resetn, w;
3   wire z;
4   fsm_n_21 DUT (.clk(clk),.resetn(resetn), .w(w), .z(z));
5   initial begin
6     w=0;
7     clk=0;
8     forever #5 clk=~clk;
9     $monitor("%t w=%d, z=%d resetn=%d", $time, w, z, resetn);
10  end
11  initial begin
12    resetn=0;
13    #10 resetn=1;
14    #10 w=0;
15    #10 w=1;
16    #10 w=0;
17    #10 w=1;
18    #10 w=1;
19    #10 w=0;
20    #10 w=1;
21    #10 w=1;
22    #10 w=1;
23    #10 w=0;
24    #10 w=1;
25    #10 $finish;
26  end
27 endmodule
```

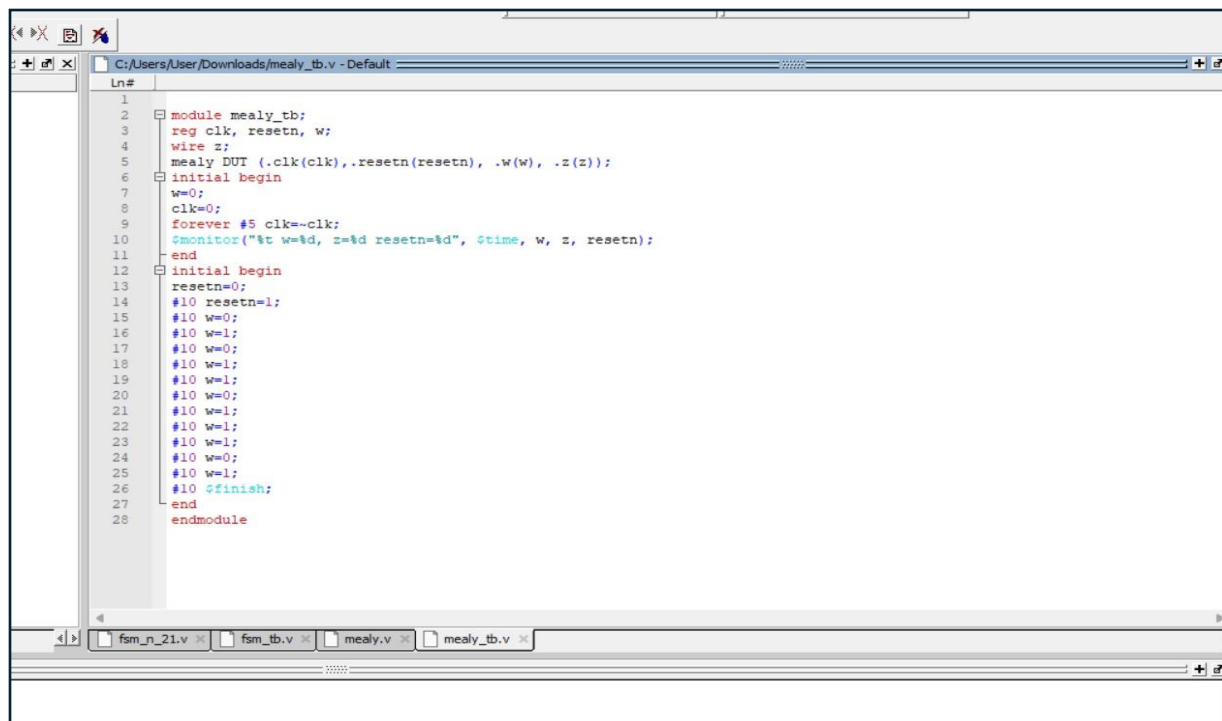
The screenshot shows a Verilog testbench code editor with the file path "C:/Users/User/Downloads/fsm_tb.v - Default". The testbench module fsm_tb instantiates the fsm_n_21 module as a DUT. It sets up a clock clk that toggles every 5 time units. The reset signal resetn is initially 0 and then transitions to 1 for 10 time units. The input w is toggled between 0 and 1 at 10-time-unit intervals. A \$monitor statement is used to observe the values of w, z, and resetn over time. The testbench ends with a \$finish statement after 10 time units.

Code for mealy machine



```
1
2 module mealy (clk, resetn, w, z);
3     input clk, resetn, w;
4     output reg z;
5
6     reg [2:1] y,Y;
7
8     parameter [2:1] A=2'b00, B=2'b01;
9
10    always@(w or y)
11    begin //NSQL
12        case(y)
13            A: if(w) Y=B;
14               else Y=A;
15            B: if(w) Y=B;
16               else Y=A;
17            default: Y=2'bxx;
18        endcase
19        if(y==B) z=1;
20        else z=0;
21    end
22    always@(negedge resetn or posedge clk)
23        if(~resetn) y<=A;
24        else y<=Y;
25    endmodule
```

Testbench code for mealy machine:



```
1
2 module mealy_tb;
3     reg clk, resetn, w;
4     wire z;
5     mealy DUT (.clk(clk),.resetn(resetn), .w(w), .z(z));
6
7     initial begin
8         w=0;
9         clk=0;
10        forever #5 clk=~clk;
11        $monitor("%t w=%d, z=%d resetn=%d", $time, w, z, resetn);
12    end
13
14    initial begin
15        resetn=0;
16        #10 resetn=1;
17        #10 w=0;
18        #10 w=1;
19        #10 w=0;
20        #10 w=1;
21        #10 w=1;
22        #10 w=0;
23        #10 w=1;
24        #10 w=0;
25        #10 w=1;
26        #10 $finish;
27    end
28 endmodule
```

Result:

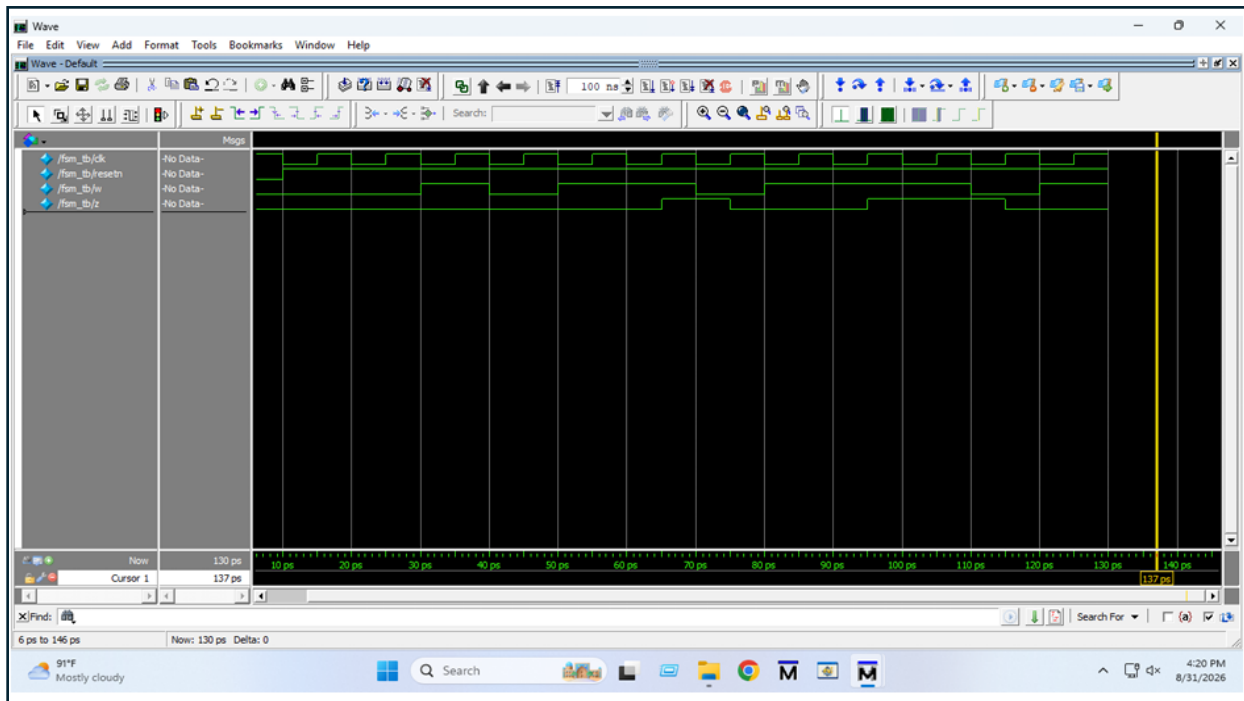


Fig : moore fsm timing diagram

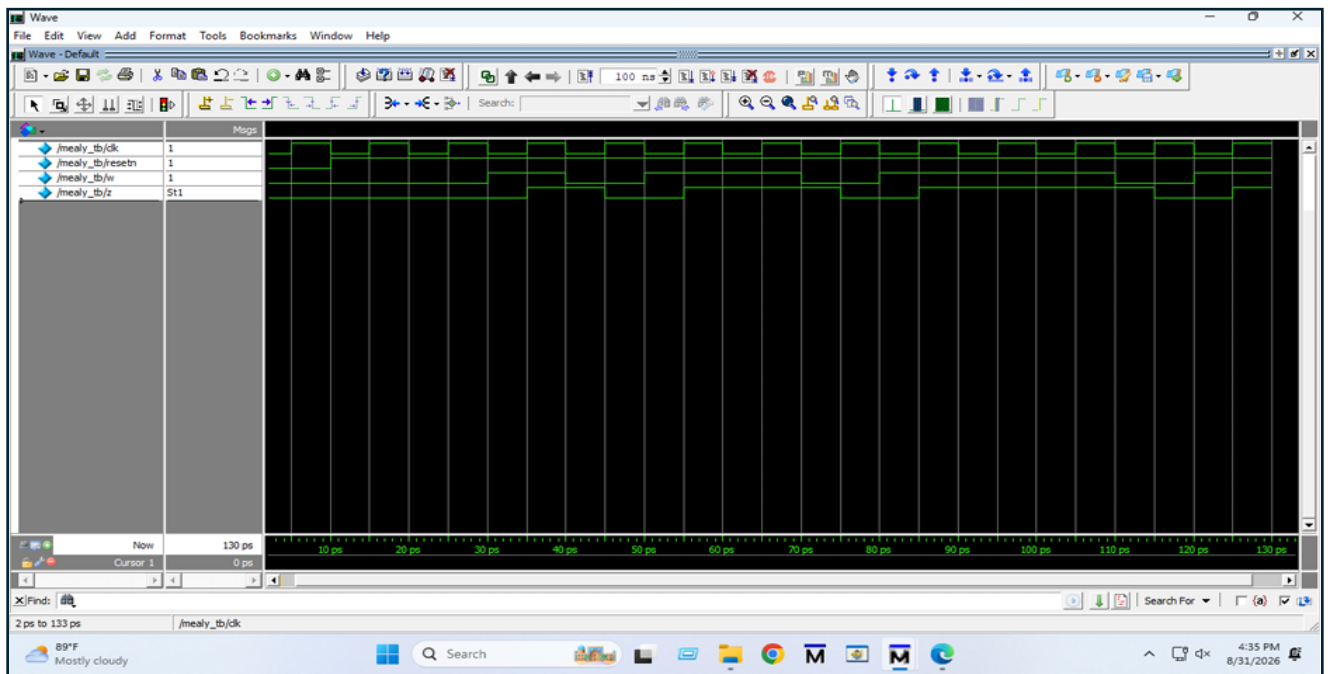


Fig : mealy fsm timing diagram

Discussion :

By comparing the two waveforms, the difference between the mealy and moore fsm becomes easier to understand. In this simulation, the Mealy machine responds when a single 1 appears in the input, while the Moore machine responds after getting the pattern 11. This happens because the two machines were designed with different state tables and state structures. One important observation is that the Mealy machine used in this experiment behaves almost like a one-clock delay, because its output mainly depends on the present state instead of directly using the input. So, even though it is a Mealy machine, its output changes with the clock, which makes its timing behavior similar to a Moore machine. The Mealy machine showed that its output is closely related to the input and state-transition conditions. As a result, its output can respond more directly to changes in the input. On the other hand, the Moore machine uses three states, A, B, and C, and the output is associated with the designated state. This makes the Moore output behavior more dependent on state changes. Overall, the simulation results matched the expected state transitions and helped verify the state logic, reset operation, clock behavior, and output response of both fsm.

Conclusion:

In this experiment, we successfully designed and simulated both mealy and moore Finite state machines using Verilog HDL in modelsim. The simulation helped us understand how the FSM changes from one state to another based on the input and clock signals. We also observed how the reset signal brings the fsm back to its initial state. The experiment made the difference between mealy and moore machines much easier to understand in a practical way. The mealy machine uses the input and present state to determine its behavior, while the Moore machine mainly depends on the present state. By writing the Verilog code and checking the waveforms in modelsim, we could clearly see how the fsm works in real simulation. Overall, this experiment gave us a good basic understanding of fsm design, state transitions, reset operation, and Verilog HDL simulation. It was also helpful in connecting the theoretical concepts of Mealy and Moore machines with their actual behavior in modelsim.